

Incident Audit: Deployment Verification Gap

SQL_TEST_01 — Ask-Your-Database Agent (Build 6) — 2026-08-07

Scope: a same-day follow-on to the 2026-08-06 token-burn incident — from the CI fix that closed that incident, through discovering the fix never reached the running browser service, to a distinct, still-open finding surfaced by the user's own re-test afterward.

Timezone: Pacific (UTC-7), converted from the container's UTC-recorded logs, consistent with the prior audit.

Evidence basis: every timestamp and token count below is pulled directly from `logs/tool_calls.jsonl`, `agent_scratch.chat_rounds`, `docker ps` output, and GitHub Actions run history — not reconstructed from memory.

Executive summary

The eval-harness fix that closed out the 2026-08-06 incident was correct and passed CI cleanly. It never reached the actual running application. The browser-facing Streamlit service had been running continuously since the previous day's last container recreate, and every verification step taken that day — repeated local test runs, a clean CI pass — used *fresh subprocesses* that re-import code from disk on every call. None of them exercised the one thing that mattered: whether the long-running server process itself had picked up the change. It hadn't. Three real user questions were answered by stale code, including one that cost **270,961 tokens** — the single largest token count logged in this project to date, worse than the original incident being fixed. The root cause was a testing-methodology gap, not a code defect: the fix was proven correct in isolation and never proven deployed. Corrected by an actual image rebuild and container recreate, confirmed by a fresh test (9,878 tokens, fully correct), and confirmed a second time independently by the user's own live re-test (15,799 tokens — a 94% reduction from the broken run). That second re-test also surfaced a distinct, unrelated finding: an open gap where the agent can answer an open-ended question from an incomplete semantic-search sample and narrate connective claims the evidence doesn't support — caught correctly by the verifier, so no wrong answer reached the user, but a real gap worth its own fix. Three same-day follow-ons closed the loop further: an automatic pre-commit guard so the documentation search index can't silently go stale again; a fix, surfaced by testing that guard end to end, for a second case where semantic search has no concept of chronological recency and can rank an older document above a genuinely newer one; and a fix for the original narrative-fabrication finding itself, generalizing the same underlying rule that closed the recency gap.

Timeline

Phase 0 — Last known-good deploy

2026-08-06, 15:20 PDT

The `askdb_streamlit` container was last rebuilt and recreated the previous day, at the end of the 2026-08-06 token-burn incident (Fix v5, the working default-rule `SYSTEM_PROMPT`). Confirmed directly: `docker ps` showed `CreatedAt: 2026-08-06 15:19:40 -0700 PDT` at the moment this new incident was diagnosed, roughly 20 hours before the fix described below.

Phase 1 — A real CI failure, correctly diagnosed and fixed

11:02 – 11:56 PDT

- A push (closing out the 2026-08-06 incident's documentation) triggered CI at 11:02:21 PDT; the `eval` job failed on a pre-existing, unrelated flaky case ("How many customers are in the database?").
- User: **"Fix the eval issue and any other. We'd like to wrap this project up cleanly."** Investigation found two real bugs, not one: the ambiguous customers question, and a stale hardcoded path in `SYSTEM_PROMPT` (`source_file = 'sql-test-01-cheatsheet.html'`, a bare filename unused since the 2026-08-05 reorg — the real value has been `docs/sql-test-01-cheatsheet.html` ever since). The bare string only ever "worked" locally because 462 orphaned pre-reorg rows were still sitting in the local database, a cleanup flagged and never completed back on 2026-08-05.
- Both fixed, verified stable across multiple fresh local runs (7/7, 5/5, 5/5 verifier cases), committed (`4d8a87f`), pushed, and confirmed via `gh run watch` — CI run `31208524967` passed fully at approximately 11:56 PDT.

Every verification step in this phase was real and passed. That's what makes the next phase worth documenting carefully — a genuinely green CI run and multiple stable local test passes still weren't sufficient evidence that the fix had reached production.

Phase 2 — Stale code answers three real questions

11:29 - 11:40 PDT

Unknown to the person fixing the issue at the time, the user was independently testing the live browser app during this exact window — before the fix's own commit had even finished its CI run. All three questions were answered by the container created the previous day, which had never re-imported any code touched today:

11:29:53	"What information is contained in your docs?"	e66197f0bce0
11:32:29	"Give ma a list of commands used"	8a680229d01f
11:40:27	"Tell me about SQL migrations" — 9 tool calls, 269,042 in / 1,919 out = 270,961 tokens	cba37f905ef6

The largest single-question token count logged in this project. The log shows exactly the bug the Phase 1 fix was meant to close: three separate attempts against the stale `source_file = 'sql-test-01-cheatsheet.html'` (returning nothing), before the model discovered the correct `docs/`-prefixed path on its own — likely from `search_docs`'s own result payload, which includes real `source_file` values — then ran three more narrow, ad hoc `LIKE`-pattern queries against the correct path instead of one direct query. Both problems compounded across all 9 turns via the same stateless-API mechanism documented in the 2026-08-06 audit.

Phase 3 — Diagnosis: the fix was correct, the deployment was not

~12:05 – 12:10 PDT

User: **"Browser app test question with results. Token count still seems extremely high."** First check: `docker exec askdb_streamlit grep source_file agent.py` — the file on disk had the fix. That result was almost misleading on its own; it confirms the file, not the running process. The real check was `docker ps --format "{{.CreatedAt}}"`, which showed the container predated every change made that day.

Root cause: every verification step taken in Phase 1 — `docker exec python -c "..."` test calls, and CI itself — spawns a fresh process that imports `agent.py` from disk on every invocation, so each one correctly picked up the fix. The actual Streamlit server is a single long-running process that imports its dependencies once, at startup. A file changing on disk after that has zero effect on it. Nothing in that day's testing had exercised the one distinction that mattered: *proving code correct* is not the same evidence as *proving a long-running service has reloaded it*.

Phase 4 — Fix and first re-verification

12:10 – 12:13 PDT

- `docker compose build streamlit_app && docker compose up -d --force-recreate streamlit_app` — a real image rebuild and container recreate, not a file patch. Confirmed fresh: `CreatedAt: 2026-08-07 12:10:28 -0700 PDT`, and the fix line present on re-check.
- The 3 chat rounds from Phase 2 removed from `agent_scratch.chat_rounds` (evidence preserved in `logs/tool_calls.jsonl`, which is append-only and untouched) — Build 6's tiered memory would otherwise have replayed that 270,961-token round into the next 3 questions' context regardless of whether the fix worked, the same risk documented in the 2026-08-06 audit.
- Re-ran the exact same question in a fresh process: **9,201 in / 677 out = 9,878 tokens**, one clean `run_sql_query` call, a complete and correct 13-migration list.

Phase 5 — Independent confirmation, and a second, distinct finding

12:13 PDT

12:13:30

"Tell me about SQL migrations" (user's own live re-test, browser) — 1
tool call, 15,408 in / 391 out = **15,799 tokens**

96cd2fb2f2e8

A 94% reduction from the broken run, confirming the deployment fix independently of the test above. But this run took a different path — `search_docs` only, no `run_sql_query` follow-up — and answered from an incomplete top-5 semantic-search sample (4 migrations, not 13), then added connective narrative the evidence never stated: that migrations are "cumulative," "idempotent," and "repeatable," and that `008_add_summary_embedding.sql` "adds vector/embedding support for equipment summaries" (the actual evidence only mentioned an unrelated fix to that file in passing).

The verifier caught this correctly, specifically, and completely — **Unverified**, citing all four unsupported claims individually rather than a generic "doesn't match." Unlike the false positive that opened the 2026-08-06 incident, this is a true positive: a real gap in the answer, correctly identified, that never reached the user unflagged. The system worked as designed here even though the underlying agent behavior — settling for a partial sample and narrating past it on open-ended "tell me about X" questions, as opposed to the more literal "list every X" phrasing `SYSTEM_PROMPT` already steers away from raw search — remains a real, separately-scoped gap, flagged here but intentionally not fixed as part of the incident itself, pending a decision on scope. (Fixed the same day — see Phase 7.)

 **Unverified:** The proposed answer makes several claims not supported by the tool evidence:

1. **Unsupported claim about migrations being cumulative and idempotent:** The evidence describes what individual migration files do and a specific gap/fix in 012_chat_rounds_schema.sql, but never states that migrations are "cumulative," "idempotent," or "repeatable" in the way described.

2. **Unsupported claim about building fresh databases:** The evidence mentions that "a genuinely fresh database gets the correct grant from one file, not a follow-up migration" in the context of fixing 012_chat_rounds_schema.sql, but this is specific to that one file's fix, not a general principle about how all migrations work.

3. **Unsupported elaboration on 008_add_summary_embedding.sql:** The evidence only mentions this file in passing as having a "CREATE EXTENSION vector fix." The proposed answer claims it "Adds vector/embedding support for equipment summaries," but the evidence doesn't specify what it adds or its purpose—it only indicates there was a fix applied to it.

4. **Overstated completeness claim:** The proposed answer ends by suggesting the cheatsheet provides "a complete, up-to-date list of all migrations used in this project," but the evidence only shows it as containing information about some migrations, not confirmation of completeness.

The evidence supports factual descriptions of specific migration files and a specific bug/fix scenario, but the broader architectural claims about how migrations work in this system go beyond what the evidence actually demonstrates.

The actual verifier output from this question (96cd2fb2f2e8), captured live in the browser — the same reasoning quoted above, unedited.

Before / after

RUN	TOKENS	PATH TAKEN	CORRECTNESS
Stale process (Phase 2)	270,961	3 failed bare-path guesses + 3 narrow re-guesses on the correct path	Eventually correct, at 27x the cost of the fixed version
Fresh subprocess, post-fix (Phase 4)	9,878	One direct, correctly-pathed query	Correct, complete (all 13 migrations)

Live browser, post-fix (Phase 5)

15,799

search_docs

only, no follow-up

Incomplete + unsupported narrative — correctly flagged
Unverified

Decision log

DECISION	WHY
Rebuild the image and force-recreate the container, not just re-verify the file on disk	The only way to actually change what a long-running process has loaded into memory; a docker cp or grep against the file proves the source is correct, not that the service is running it.
Remove the 3 stale-process chat rounds from persisted history before re-testing	Build 6's tiered memory replays the last 3 rounds in full into every new question's context — leaving them in would have re-cost their token weight regardless of whether the underlying fix worked, the same risk already documented and handled the same way in the 2026-08-06 incident.
Document the search_docs narrative-fabrication finding without fixing it in the same pass	It's a distinct problem from the deployment gap this incident is actually about, correctly caught by the verifier rather than reaching the user silently, and scoping it as its own decision rather than bundling an unrelated fix into an incident report.

Lessons

- A fresh subprocess test and a long-running service are not interchangeable evidence. `docker exec python -c "..."` proves code correctness; only an actual restart of the service being tested proves deployment. Both are needed, and neither substitutes for the other.
- CI passing is real evidence the code is correct in a clean environment — it says nothing about whether a separately-running local or production service has picked up that same code, since CI always builds its own container from scratch.
- The most convincing-looking check ("the file on disk has the fix") can be the wrong one. The right question wasn't "does the source contain the change" but "when was the process that's actually serving requests last started."
- A verifier correctly flagging a bad answer is a successful outcome, not an incident on its own — Phase 5 is worth documenting as a finding, but it's evidence Build 7 is doing its job, not a new failure of the system as a whole.

Phase 6 — Two follow-on fixes, same day

6a — an automatic guard against the index going stale again

Asked directly: "Can this update process be automatic?" — referring to `docs.chunks` needing a manual `extract_doc_chunks.py` + `embed_chunks.py` re-run after any doc edit, the same class of gap (data this time, not code) as the deployment gap earlier in this incident.

Chosen fix, over two lighter alternatives (a single combined convenience script; a startup staleness warning): a git pre-commit hook, tracked in the repo (`.githooks/pre-commit` , wired in via `git config core.hooksPath .githooks` — not the untracked `.git/hooks/` , which never survives a clone) that re-runs the extract/embed pair whenever a commit touches `docs/sql-test-01-cheatsheet.html` , `docs/project-overview.html` , or any handoff brief. Runs on the host directly (confirmed the host `.venv` already has every required package, and `pg_local` is reachable at `127.0.0.1:5432`) — no container involved. Blocks the commit on failure, the same strictness CI already applies, with `--no-verify` as the standard escape hatch. Tested directly (staged a real pending doc change, ran the hook without committing): correctly detected the change, refreshed 522 chunks, confirmed the 2026-08-07 incident content was queryable immediately afterward.

6b — testing the fix immediately surfaced the finding Phase 5 had left open

Re-running "what was the latest incident reported in the docs" against the now-fresh index still returned **2026-08-06**, not 2026-08-07 — confirmed the new chunk was indexed (a direct query found it), so this wasn't the staleness bug recurring. `search_docs` ranks by embedding similarity, which has no concept of chronological order; two thematically similar incident callouts don't rank by date just because one is newer. This is the same category of gap Phase 5 flagged and left open (settling for a search result that isn't actually the right kind of answer for the question asked) — "latest/most recent X" needed the identical treatment already given to "list every X."

Fix: `SYSTEM_PROMPT` extended to steer "latest/most recent X" documentation questions toward `run_sql_query` with `ORDER BY chunk_id DESC` , alongside the existing "list every X" guidance — correct specifically because the cheat sheet is append-only (new entries added at the end, never edited in place), so a higher `chunk_id` within that one file reliably means "added more recently." Deployed via a real `docker compose build + up -d --force-recreate` , confirmed fresh via `CreatedAt` before testing — applying Phase 3's own lesson immediately rather than repeating the mistake it describes. Re-tested: 2026-08-07 correctly identified as the latest incident, both incidents correctly ordered, 10,064 tokens. Regression-checked against the full eval suite (7/7, 5/5) before considering it done.

Phase 7 — closing Phase 5's finding: grounding search_docs against incomplete samples

The narrative-fabrication gap flagged in Phase 5 and left open — an open-ended question answered from an incomplete `search_docs` sample, then narrated past what the evidence actually showed — got the same treatment as Phase 6b's fix, generalized further. Root cause common to all three fixes in this incident: a keyword-phrasing trigger ("list every X", then "latest/most recent X") is too narrow, because the real signal isn't the phrasing, it's whether the question is about a category the cheat sheet already tracks as a complete table (every file, every command, every SQL migration) regardless of how it's asked - "tell me about the SQL migrations" needs the same query as "list every SQL migration," even though only one of those literally says "list."

Fix: `SYSTEM_PROMPT` 's SQL-routing rule widened from matching specific phrasings to matching the underlying category - file/command/migration questions route to `run_sql_query` regardless of phrasing. Separately, added an explicit grounding rule for the cases where `search_docs` genuinely is the right tool (narrative/thematic questions with no enumerable category behind them): state only what the retrieved chunks actually say, don't generalize a pattern across the whole system unless a chunk states it explicitly, and say what's missing rather than filling a partial sample with something plausible-sounding.

Re-tested the exact original question ("Tell me about SQL migrations") on a freshly rebuilt and recreated container: this time `run_sql_query` retrieved all 13 migrations in one call (vs. the original 4-migration sample), and the answer's claims all trace directly to retrieved evidence - verified `supported: true`, with the verifier's reasoning specifically confirming each claim against its source chunk rather than a generic pass. 6,959 in / 835 out = 7,794 tokens. Regression-checked against the full eval suite and the verifier eval harness (7/7, 5/5, 5/5) before considering it done.

Cross-incident comparison: token burn, before and after

Pulling every real logged number across both 2026-08-06 and this incident into one view:

QUESTION	BEFORE	AFTER	REDUCTION	CORRECTNESS
"How many different types of cameras?" (2026-08-06)	177,090	8,024	95.5%	Wrong (4 types) → correct (5)
"How many laptops do you have?" (2026-08-06)	45,469	6,018	86.8%	Wrong (1,501) → correct (482)
"How many types of microphones?" (2026-08-06)	14,801	7,434	49.8%	Correct, false-negative badge → correct + verified

"Tell me about SQL migrations" (2026-08-07)	270,961	7,794	97.1%	Wrong + incomplete → correct, every claim traced to evidence
--	---------	-------	-------	---

"Before" is the worst real logged run for each question, not a hypothetical baseline; "after" is the final, browser- or verifier-confirmed state, not an intermediate fix attempt.

The throughline: every fix that actually held converged on the same shape - resolve the ambiguity once, as data or as a stated default rule, not as a heuristic re-derived from scratch on every question. The fixes that didn't hold on the first attempt were exactly the ones still asking the model to infer something from context rather than check a fact: the cheap-but-wrong aggregate query (2026-08-06 v1, 11,926 tokens, mislabeled an accessory), and the narrow phrasing-based patches here (2026-08-07, fixed twice before the underlying category-vs-phrasing rule was actually correct). Cost and correctness moved together every time, not against each other - the cheapest wrong answer was never actually cheaper than the eventual correct one, once what it would take to get the model to argue itself out of a bad guess is counted too.